

RANK-OPTIMAL DIRECTIONAL TAYLOR JETS FOR HIGH-ORDER MIXED DERIVATIVES OF NEURAL SURROGATES

AUTHOR ONE* AND AUTHOR TWO†

Abstract. Evaluating one prescribed high-order mixed partial derivative of a neural surrogate is a recurring cost in scientific computing, and the price of an evaluation is set by how many passes through the model it takes. Repeated automatic differentiation is exact but nests, numerical differentiation trades exactness for a step size, and randomized estimators trade it for variance. Taylor-mode automatic differentiation removes the nesting by propagating truncated Taylor coefficients along one direction at a time, which leaves open how many directions a prescribed derivative needs. We settle that question. For a fixed order the directional Taylor coefficients form a homogeneous polynomial in the direction, so extracting one mixed derivative extracts one of its coefficients, and a directional formula is a decomposition of a monomial into powers of linear forms. The shortest formula therefore has as many terms as the Waring rank of that monomial, which is known in closed form over the complex numbers and is attained by a roots-of-unity construction that requires no linear solve. Realized as one batched Taylor jet with a closed-form adjoint, the schedule becomes a derivative backend that is algebraically exact, differentiable in the model parameters, and able to take complex directions. Double-precision tests reproduce nested automatic differentiation for values and for parameter gradients. In training runs where the derivative backend is the only quantity that varies, the schedule reduces the time and the memory of one optimizer step by a factor that grows with the derivative order, and the saved time converts into lower error on the higher-order problems.

Key words. high-order derivatives, Taylor-mode automatic differentiation, Waring decomposition, apolarity, complex-valued neural networks, physics-informed neural networks

AMS subject classifications. 65D25, 68T07, 14N07, 35J40

1. Introduction. Derivatives of a neural surrogate with respect to its inputs are needed wherever a learned function has to satisfy, or be probed against, a differential relation. Neural representations of the solution of a partial differential equation are trained by penalizing the residual of that equation, so every optimization step differentiates the network at many points [6, 12, 15]. Variational Monte Carlo evaluates a differential operator applied to a neural wavefunction at every sampled configuration [14]. In each case the differentiation sits inside the innermost loop, is repeated for the whole of training, and is applied to the same model at many points at once, so its cost is a first-order concern rather than an implementation detail. The concern sharpens with the order of the derivative: a fourth- or sixth-order operator is far more expensive to evaluate than a gradient, and the gap widens with the depth of the model.

The exact ways of computing such a derivative are long established, and each pays for exactness differently. Symbolic differentiation returns a closed-form expression whose size grows quickly with the order. Numerical differentiation evaluates the model on a stencil and is indifferent to its internal structure, but it introduces a step size, and truncation and cancellation errors that degrade as the order rises; coupling it with exact differentiation recovers part of the accuracy while keeping the locality [3]. Automatic differentiation avoids both drawbacks by differentiating the program itself [5]. Its reverse mode is the standard route to a gradient, and a high-order derivative is usually obtained by applying it repeatedly; each application, however, differentiates the graph built by the previous one, so depth and storage grow with the order. Taylor mode avoids that nesting: it propagates truncated Taylor

*Institution One (author1@example.com).

†Institution Two (author2@example.com).

coefficients through every primitive, and one forward pass along a chosen direction returns the directional derivatives of all orders up to the truncation [1, 5]. The cost of Taylor mode is therefore counted in passes, one per direction, which makes the number of directions the quantity worth minimizing.

Where exactness can be surrendered, estimation is cheaper still, and a substantial literature now trades it for variance. Hutchinson-type estimators replace the trace of a derivative tensor by an expectation of quadratic forms, so that a Laplacian costs Hessian-vector products rather than a Hessian [8]. Dimension-sampling estimators evaluate a random subset of the coordinate terms of the operator at each step [10]. Randomized smoothing convolves the model with a Gaussian and uses Stein identities to express derivatives of the smoothed function as expectations of function values, removing the differentiation entirely at the price of a smoothing bandwidth and of a variance that grows as the bandwidth shrinks [7, 9, 17]. The stochastic Taylor derivative estimator generalizes the idea to an arbitrary contraction of a derivative tensor by pushing forward randomly drawn sparse jets [16]. These methods are unbiased and scale to dimensions that exact evaluation cannot reach, but the estimate they return is exact only in expectation, and the sampling budget reappears in the accuracy of whatever is trained on it.

A third line keeps exactness and buys speed from the structure of one particular operator. The forward Laplacian propagates the value, the gradient, and the Laplacian together through the network, exploiting the way a trace collapses through a linear layer, and is now standard in variational Monte Carlo [4, 14]. Specializations of this kind are exact and fast, but each is tied to the operator it was designed for.

What is missing from all three lines is an answer for the elementary exact case: one prescribed mixed partial derivative, computed deterministically, of a model that can be evaluated along any direction. Taylor mode supplies the derivative along whichever direction it is given, and it is silent about how many directions the target requires. One can of course write down formulas: a mixed derivative is recoverable from directional evaluations along suitable combinations of coordinate directions, and such reconstructions are elementary to produce. What is not elementary is knowing when one has finished, because a formula obtained by hand carries no certificate that fewer evaluations would not do. The question we ask is therefore the sharp one: for a given mixed derivative, what is the smallest number of directional evaluations that reconstructs it exactly, and which directions attain that minimum?

This paper answers that question and turns the answer into an implementation. The route is to stop thinking of the directions as a stencil and to look at what the directional Taylor coefficients are as a function of the direction: for a fixed order they form a homogeneous polynomial, and extracting one mixed derivative is extracting one of its coefficients. A directional formula is then a decomposition of a monomial into powers of linear forms, so the shortest formula has as many terms as the Waring rank of that monomial, a quantity the algebraic-geometry literature settled in closed form. That rank is known in closed form over the complex numbers, the minimizing directions are roots of unity written down without solving anything, and the answer is a minimum rather than an upper bound, so it also says when no further saving is available. Because the construction is exact rather than approximate, it composes with the training loop in the ordinary way: we evaluate the whole schedule in one batched Taylor jet through an entire activation, and give the jet a closed-form adjoint so that the result stays differentiable in the model parameters.

The contributions of this work are as follows:

- (a) We prove that directional formulas for a fixed mixed derivative correspond exactly to Waring decompositions of the associated monomial, so that the minimum number of directional evaluations is a Waring rank, and we give a schedule that attains it in closed form.
- (b) We realize that schedule as a single batched Taylor jet with a closed-form adjoint, obtaining a derivative backend that is algebraically exact, differentiable in the model parameters, admits complex directions natively, and agrees with nested automatic differentiation in double precision.
- (c) We measure the backend inside complete training runs in which the derivative evaluation is the only thing that varies, and report how the saving in step time, in memory, and in the accuracy reached under a fixed budget grows with the derivative order.

The remainder of this paper is organized as follows. Section 2 fixes the evaluation problem and reviews Taylor-mode evaluation of directional derivatives. Section 3 identifies the shortest directional formula with a monomial Waring decomposition, states and proves the main theorem, and gives the resulting algorithm. Section 4 verifies the implementation against nested automatic differentiation and then demonstrates it inside complete training runs on three benchmark families. Section 5 concludes. The roots-of-unity filter, the explicit schedules used in the experiments, and the apolar form of the rank bound are collected in Appendix A.

2. Preliminaries.

2.1. The evaluation problem. Let $u_\theta: \mathbb{R}^d \rightarrow \mathbb{R}$ be a neural surrogate with parameters θ , smooth in its inputs. Let $\nu = (\nu_1, \dots, \nu_d) \in \mathbb{Z}_{\geq 0}^d$ be a multi-index of order $p = |\nu|$, and write $\partial^\nu := \partial_1^{\nu_1} \cdots \partial_d^{\nu_d}$ for the associated derivative. The task addressed in this paper is the evaluation of the single mixed partial derivative $\partial^\nu u_\theta$ at a given point, subject to three requirements that together rule out most of the alternatives surveyed in Section 1. The value must be exact, so that no step size and no sampling budget enters the result. The multi-index is fixed and prescribed, rather than being contracted against a structured family such as a trace, so an operator-specific specialization does not apply. And the value must remain differentiable in θ , since it is consumed by an optimizer that updates the surrogate. The derivative is also evaluated at many points at once and re-evaluated at every step, so what matters is the cost of one evaluation of one derivative, amortized over a batch.

The benchmarks of Section 4 instantiate this task in its most demanding common form, the residual of a boundary-value problem. Let $\Omega \subset \mathbb{R}^d$ be a bounded domain with boundary $\Gamma := \partial\Omega$, and consider

$$\mathcal{P}u = f \quad \text{in } \Omega, \quad \mathcal{B}u = g \quad \text{on } \Gamma, \quad (2.1)$$

with $\mathcal{P}$ a differential operator of order p and $\mathcal{B}$ the boundary conditions. Given interior points $(\mathbf{x}_{r,i})_{i=1}^{n_r} \subset \Omega$ and boundary points $(\mathbf{x}_{b,i})_{i=1}^{n_b} \subset \Gamma$, a physics-informed network fits (2.1) by minimizing

$$\mathcal{L}(\theta) := \frac{\lambda_r}{n_r} \sum_{i=1}^{n_r} |\mathcal{P}u_\theta(\mathbf{x}_{r,i}) - f(\mathbf{x}_{r,i})|^2 + \frac{\lambda_b}{n_b} \sum_{i=1}^{n_b} |\mathcal{B}u_\theta(\mathbf{x}_{b,i}) - g(\mathbf{x}_{b,i})|^2. \quad (2.2)$$

Expanding $\mathcal{P}$ into monomial terms turns one evaluation of the residual in (2.2) into several instances of the task above, on the same surrogate and the same batch of points; a polyharmonic operator $\mathcal{P} = \Delta^m$, for example, expands into $\binom{d+m-1}{m}$ of

them. The loss (2.2) is a consumer of the derivative and not the object of study: nothing in what follows is specific to it.

2.2. Taylor-mode evaluation of directional derivatives. Let $u: \mathbb{R}^d \rightarrow \mathbb{R}$ be sufficiently smooth near a fixed point $\mathbf{x}$. For a direction $\mathbf{v} \in \mathbb{R}^d$, the directional Taylor coefficient of order p at $\mathbf{x}$ along $\mathbf{v}$ is

$$T_p(\mathbf{x}; \mathbf{v}) := \frac{1}{p!} \frac{d^p}{d\tau^p} u(\mathbf{x} + \tau \mathbf{v}) \Big|_{\tau=0} = \frac{1}{p!} D^p u(\mathbf{x})[\mathbf{v}, \dots, \mathbf{v}]. \quad (2.3)$$

Taylor-mode automatic differentiation evaluates (2.3) in one forward pass [1, 5]. Each intermediate activation $y \in \mathbb{K}^B$ over a batch of B points, with $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, is replaced by the length- $(p+1)$ tuple $(y_0, \dots, y_p)$ of its truncated Taylor coefficients $y_k = (1/k!) d^k y / d\tau^k|_{\tau=0}$. For a multilayer perceptron $u = \phi_L \circ A_L \circ \dots \circ \phi_1 \circ A_1$ with affine maps A_ℓ and entrywise activations ϕ_ℓ , an affine layer $y = Wx + b$ propagates each order independently,

$$y_0 = Wx_0 + b, \quad y_k = Wx_k \quad (k \geq 1), \quad (2.4)$$

and each nonlinearity is propagated together with one auxiliary jet that closes its derivative recurrence. For $\tanh$ the auxiliary jet is $z = 1 - y^2$, initialized by $y_0 = \tanh(x_0)$ and $z_0 = 1 - y_0^2$, and

$$y_k = \frac{1}{k} \sum_{j=0}^{k-1} (k-j) z_j x_{k-j}, \quad z_k = - \sum_{j=0}^k y_j y_{k-j}, \quad k \geq 1. \quad (2.5)$$

For $\sinh$ the auxiliary jet is $z = \cosh(x)$, initialized by $y_0 = \sinh(x_0)$ and $z_0 = \cosh(x_0)$, and

$$y_k = \frac{1}{k} \sum_{j=0}^{k-1} (k-j) z_j x_{k-j}, \quad z_k = \frac{1}{k} \sum_{j=0}^{k-1} (k-j) y_j x_{k-j}, \quad k \geq 1. \quad (2.6)$$

The recurrence (2.6) has the same coupled form as the one for $\sin$ and $\cos$, except that the $\cosh$ update carries no sign flip. The implementation also contains the recurrences for $\sin$ and for the logistic sigmoid, which the benchmark architectures of Section 4 require. Every coefficient y_k is an ordinary tensor, and the propagation stores $p+1$ coefficient tensors per activation without nesting input-gradient graphs.

3. Rank-optimal directional schedules. The argument of this section is short, and it is worth naming its four steps before the details arrive. First, the directional Taylor coefficients of order p , read as a function of the direction rather than of the point, are the coefficients of a homogeneous polynomial of degree p , and the target derivative is one of them. Second, a formula that reconstructs the derivative from finitely many directional evaluations is therefore a linear identity among such coefficients, which reduces the design problem to a finite system of moment conditions. Third, pairing each direction with the polynomial variable turns that system into a single polynomial identity: the monomial belonging to the target derivative, written as a linear combination of p th powers of linear forms. Fourth, the least number of terms in such a combination is by definition the Waring rank of that monomial, a quantity known in closed form, and one set of minimizing linear forms consists of roots of unity. The four steps together replace a search for a stencil

by a rank computation, and because the outcome is a minimum, it certifies that no shorter formula exists.

We introduce some notation used throughout this section. Let $\mathbb{N}_0 := \{0, 1, 2, \dots\}$, and let $\mathbb{C}^d$ be the d -dimensional complex space. The *active coordinates* of ν are the indices k with $\nu_k > 0$; we relabel them as $i_0, \dots, i_n$ and set $a_j := \nu_{i_j}$, ordered so that $1 \leq a_0 \leq \dots \leq a_n$, with ties broken by the original coordinate index. The ordering singles out a coordinate i_0 of least exponent, which is the coordinate the rank formula treats separately. We write $a := (a_0, \dots, a_n)$, and we record for later use that

$$p = \sum_{j=0}^n a_j, \quad \nu! = \prod_{k=1}^d \nu_k! = \prod_{j=0}^n a_j!, \quad m_j := a_j + 1. \quad (3.1)$$

The *active subspace* is $V := \text{span}\{e_{i_0}, \dots, e_{i_n}\} \subset \mathbb{R}^d$, and $V_{\mathbb{C}} := V \otimes_{\mathbb{R}} \mathbb{C}$ is its complexification. Since ∂^ν is insensitive to coordinates outside the active set, all directional probes below may be taken in V or in $V_{\mathbb{C}}$. For $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, the space of homogeneous polynomials of degree p in $\mathbf{z} = (z_0, \dots, z_n)$ is denoted $\mathbb{K}[\mathbf{z}]_p$, and $[\mathbf{z}^a]F$ denotes the coefficient of $\mathbf{z}^a = z_0^{a_0} \dots z_n^{a_n}$ in F . For $m \geq 1$, let $U_m := \{\zeta \in \mathbb{C} : \zeta^m = 1\}$, and let $\mathbf{1}_A$ be the indicator of a condition A .

The construction of this section uses complex directions, so we fix their meaning once. For $\mathbf{v} \in V_{\mathbb{C}}$ we define $T_p(\mathbf{x}; \mathbf{v})$ by the complex multilinear extension of $D^p u(\mathbf{x})$ on the right-hand side of (2.3). When u admits a holomorphic extension to a neighbourhood of $\mathbf{x}$ in $\mathbb{C}^d$, this definition agrees with the order- p coefficient of $\tau \mapsto u(\mathbf{x} + \tau \mathbf{v})$, and the recurrences (2.4)–(2.6) evaluate it on complex tensors. In practice the extension is guaranteed by entire activations such as $\sinh(z) = (e^z - e^{-z})/2$; see Remark 3.4. The distinction matters because an arbitrary C^p function on $\mathbb{R}^d$ need not be defined at complex arguments.

3.1. Basic expansion. Let $\mathbf{z} \in \mathbb{C}^{n+1}$ parametrize the active subspace through $\mathbf{v}(\mathbf{z}) := \sum_{j=0}^n z_j e_{i_j}$, and define the *generating polynomial* of u at $\mathbf{x}$ by

$$Q_u(\mathbf{z}) := T_p(\mathbf{x}; \mathbf{v}(\mathbf{z})). \quad (3.2)$$

The symmetric p -linearity of $D^p u(\mathbf{x})$ and the multinomial theorem give

$$D^p u(\mathbf{x})[\mathbf{v}(\mathbf{z}), \dots, \mathbf{v}(\mathbf{z})] = \sum_{|\beta|=p} \binom{p}{\beta} \mathbf{z}^\beta \partial^\beta u(\mathbf{x}) = \sum_{|\beta|=p} \frac{p!}{\beta!} \mathbf{z}^\beta \partial^\beta u(\mathbf{x}),$$

where β ranges over multi-indices on the active variables and $\partial^\beta := \partial_{i_0}^{\beta_0} \dots \partial_{i_n}^{\beta_n}$. Dividing by $p!$ and comparing with (2.3) and (3.2) yields

$$Q_u(\mathbf{z}) = \sum_{|\beta|=p} \frac{\partial^\beta u(\mathbf{x})}{\beta!} \mathbf{z}^\beta, \quad (3.3)$$

so Q_u is homogeneous of degree p and its coefficients are the order- p derivatives of u at $\mathbf{x}$. Reading off the coefficient of $\mathbf{z}^a$ in (3.3) and using $a! = \nu!$ from (3.1) gives

$$\partial^\nu u(\mathbf{x}) = \nu! [\mathbf{z}^a] Q_u(\mathbf{z}). \quad (3.4)$$

Extracting one mixed derivative is therefore the extraction of one coefficient of a homogeneous polynomial of degree p .

The evaluations available to us are values of Q_u at individual directions, so we ask for the shortest linear combination of such values that realizes the functional (3.4). A *directional schedule* of length R for ∂^ν over $\mathbb{K}$ is a finite collection $(c_r, \mathbf{v}_r)_{r=1}^R \subset \mathbb{K} \times V_{\mathbb{K}}$ such that

$$\partial^\nu u(\mathbf{x}) = \sum_{r=1}^R c_r T_p(\mathbf{x}; \mathbf{v}_r) \quad (3.5)$$

for every u that is C^p on a neighbourhood of $\mathbf{x}$, and, when $\mathbb{K} = \mathbb{C}$, holomorphic there. Because (3.5) is required for all such u , the order- p derivatives $(\partial^\beta u(\mathbf{x}))_{|\beta|=p}$ may be prescribed independently: for any coefficients q_β , the polynomial $\sum_\beta q_\beta \prod_j (y_{i_j} - x_{i_j})^{\beta_j}$ has $\partial^\beta u(\mathbf{x}) = \beta! q_\beta$. Substituting (3.3) into the right-hand side of (3.5) gives

$$\sum_{r=1}^R c_r T_p(\mathbf{x}; \mathbf{v}_r) = \sum_{|\beta|=p} \frac{\partial^\beta u(\mathbf{x})}{\beta!} \left(\sum_{r=1}^R c_r \mathbf{v}_r^\beta \right), \quad (3.6)$$

and comparing (3.6) with the target $\partial^\nu u(\mathbf{x}) = \partial^a u(\mathbf{x})$ coefficient by coefficient shows that (3.5) holds for every admissible u if and only if

$$\sum_{r=1}^R c_r \mathbf{v}_r^\beta = \nu! \delta_{a\beta}, \quad |\beta| = p, \quad (3.7)$$

where $\delta_{a\beta} := 1$ if $\beta = a$ and $\delta_{a\beta} := 0$ otherwise. The design problem is thus to solve the moment conditions (3.7) with as few directions as possible.

Two features of (3.7) decide how short a schedule can be. The conditions number $\binom{p+n}{n}$, so their count is governed by how many coordinates the derivative touches and not by the order alone: a derivative of order six on three coordinates imposes far fewer conditions than one of order six on six coordinates. They are also nonlinear in the unknowns, since each direction enters through the monomials $\mathbf{v}_r^\beta$, which is why the minimum is not read off by counting equations and unknowns. Section 3.2 resolves both by recognizing (3.7) as a polynomial identity whose shortest solution is classical.

3.2. Minimal schedules and monomial Waring rank. Pair a direction $\mathbf{v} \in V_{\mathbb{C}}$, written $\mathbf{v} = \sum_{j=0}^n v_j e_{i_j}$ in active coordinates, with the polynomial variable $\mathbf{z}$ through $\mathbf{v} \cdot \mathbf{z} := \sum_{j=0}^n v_j z_j$. Expanding the p th power of this pairing by the multinomial theorem gives

$$\sum_{r=1}^R c_r (\mathbf{v}_r \cdot \mathbf{z})^p = \sum_{|\beta|=p} \binom{p}{\beta} \left(\sum_{r=1}^R c_r \mathbf{v}_r^\beta \right) \mathbf{z}^\beta, \quad (3.8)$$

whose coefficients are the very quantities constrained by (3.7). The moment conditions are therefore a polynomial identity in disguise, and the shortest schedule is the shortest decomposition of a monomial into pure powers of linear forms. That notion has a classical name.

DEFINITION 3.1 (Waring rank). *Let $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$ and let $F \in \mathbb{K}[\mathbf{z}]_p$ be homogeneous of degree p . The Waring rank $R_{\mathbb{K}}(F)$ is the least R for which there exist $c_r \in \mathbb{K}$ and linear forms $L_r \in \mathbb{K}[\mathbf{z}]_1$ with $F = \sum_{r=1}^R c_r L_r(\mathbf{z})^p$.*

Since $\mathbb{R} \subset \mathbb{C}$, every real decomposition is a complex one, so $R_{\mathbb{C}}(F) \leq R_{\mathbb{R}}(F)$, and enlarging the field can only shorten a schedule. The following theorem is the

main result of the paper. Its three parts identify the design problem with a Waring decomposition, evaluate the resulting minimum, and exhibit a schedule of that length in closed form.

THEOREM 3.2 (Rank-optimal directional schedule). *Let ν be a multi-index of order $p \geq 1$ with active exponents $1 \leq a_0 \leq \dots \leq a_n$ and $m_j := a_j + 1$ as in (3.1), and let $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$.*

- (i) *A collection $(c_r, \mathbf{v}_r)_{r=1}^R \subset \mathbb{K} \times V_{\mathbb{K}}$ is a directional schedule for ∂^ν if and only if*

$$p! \mathbf{z}^a = \sum_{r=1}^R c_r (\mathbf{v}_r \cdot \mathbf{z})^p \quad \text{in } \mathbb{K}[\mathbf{z}]_p. \quad (3.9)$$

- (ii) *The minimum length of a directional schedule for ∂^ν over $\mathbb{K}$ equals $R_{\mathbb{K}}(\mathbf{z}^a)$, and over $\mathbb{C}$ this minimum is*

$$R_{\mathbb{C}}(\mathbf{z}^a) = \prod_{j=1}^n m_j = \frac{\prod_{j=0}^n m_j}{m_0}. \quad (3.10)$$

- (iii) *For $\zeta = (\zeta_1, \dots, \zeta_n) \in U_{m_1} \times \dots \times U_{m_n}$, put*

$$\mathbf{v}_\zeta := e_{i_0} + \sum_{j=1}^n \zeta_j e_{i_j}, \quad c_\zeta := \frac{\nu!}{\prod_{j=1}^n m_j} \prod_{j=1}^n \zeta_j. \quad (3.11)$$

Then $(c_\zeta, \mathbf{v}_\zeta)_\zeta$ is a directional schedule for ∂^ν of length $\prod_{j=1}^n m_j$, and by part (ii) no complex schedule is shorter.

Proof.

(i) The moment conditions are a polynomial identity. By (3.7), the collection is a directional schedule if and only if $\sum_r c_r \mathbf{v}_r^\beta = \nu! \delta_{a\beta}$ for every β with $|\beta| = p$. By (3.8), the identity (3.9) holds if and only if $\binom{p}{\beta} \sum_r c_r \mathbf{v}_r^\beta = p! \delta_{a\beta}$ for every such β . Dividing the second family of conditions by $\binom{p}{\beta}$ turns it into $\sum_r c_r \mathbf{v}_r^\beta = \beta! \delta_{a\beta}$, and $\beta! \delta_{a\beta} = \nu! \delta_{a\beta}$ by (3.1). The two families coincide, which proves part (i).

(ii) The minimum length is a Waring rank. Absorbing the nonzero constant $p!$ into the coefficients c_r shows that a decomposition of $p! \mathbf{z}^a$ into R pure p th powers exists if and only if one of $\mathbf{z}^a$ does, so $R_{\mathbb{K}}(p! \mathbf{z}^a) = R_{\mathbb{K}}(\mathbf{z}^a)$. Combining this with part (i) identifies the minimum schedule length over $\mathbb{K}$ with $R_{\mathbb{K}}(\mathbf{z}^a)$. The value (3.10) of the complex rank of a monomial is the theorem of Carlini, Catalisano, and Geramita [2]; Appendix A.3 recalls the apolarity argument behind its lower bound.

(iii) The roots-of-unity schedule attains the minimum. We use the conventions that an empty product equals 1 and that a Cartesian product over an empty index set has one element, the empty tuple; these cover the case $n = 0$, in which (3.11) produces the single direction e_{i_0} with coefficient $\nu!$. By part (i) it suffices to prove the identity $\sum_\zeta c_\zeta (\mathbf{v}_\zeta \cdot \mathbf{z})^p = p! \mathbf{z}^a$. Fix β with $|\beta| = p$ and let C_β denote the coefficient of $\mathbf{z}^\beta$ on the left-hand side. Inserting (3.11) and applying the multinomial theorem gives

$$C_\beta = \frac{\nu!}{\prod_{j=1}^n m_j} \binom{p}{\beta} \sum_{\zeta_1 \in U_{m_1}} \dots \sum_{\zeta_n \in U_{m_n}} \prod_{j=1}^n \zeta_j^{\beta_j+1} = \frac{\nu!}{\prod_{j=1}^n m_j} \binom{p}{\beta} \prod_{j=1}^n \left(\sum_{\zeta_j \in U_{m_j}} \zeta_j^{\beta_j+1} \right), \quad (3.12)$$

where the second equality separates the independent sums. By Lemma A.1, the j th factor of the product in (3.12) vanishes unless $m_j \mid \beta_j + 1$, and $m_j = a_j + 1$ turns this divisibility into the congruence $\beta_j \equiv a_j \pmod{m_j}$. Suppose $C_\beta \neq 0$, and write $\beta_j = a_j + t_j m_j$ with $t_j \in \mathbb{N}_0$ for $j = 1, \dots, n$. The degree constraint $|\beta| = p = \sum_{j=0}^n a_j$ then forces $\beta_0 = a_0 - \sum_{j=1}^n t_j m_j$. If some t_j were positive, then $m_j = a_j + 1 \geq a_0 + 1$ by the ordering of the exponents, whence $\beta_0 \leq a_0 - m_j < 0$, contradicting $\beta_0 \geq 0$. Every t_j therefore vanishes, so $\beta_j = a_j$ for $j \geq 1$, and the degree constraint gives $\beta_0 = a_0$ as well. Hence $C_\beta = 0$ for every $\beta \neq a$. For $\beta = a$ each inner sum in (3.12) equals m_j , the product cancels the prefactor, and

$$C_a = \nu! \binom{p}{a} = \left(\prod_{j=0}^n a_j! \right) \frac{p!}{\prod_{j=0}^n a_j!} = p!.$$

The left-hand side of the identity therefore equals $p! z^a$, and the schedule has $|U_{m_1} \times \dots \times U_{m_n}| = \prod_{j=1}^n m_j$ directions. $\square$

Theorem 3.2 converts a question about derivative evaluation into a rank computation whose answer is a product of the incremented exponents, and it delivers the minimizing directions without solving a linear system: the coefficients in (3.11) are roots of unity scaled by $\nu! / R_C(z^a)$, so their magnitudes are known in advance. Three consequences deserve separate mention.

REMARK 3.3 (The smallest instance). *For $\partial^\nu = \partial_1 \partial_2$ we have $n = 1$, $a_0 = a_1 = 1$, $m_1 = 2$, and $U_2 = \{+1, -1\}$, so the roots of unity are real and (3.11) returns the two pairs $(\frac{1}{2}, e_1 + e_2)$ and $(-\frac{1}{2}, e_1 - e_2)$, that is, the elementary identity*

$$\partial_1 \partial_2 u(\mathbf{x}) = \frac{1}{2} T_p(\mathbf{x}; e_1 + e_2) - \frac{1}{2} T_p(\mathbf{x}; e_1 - e_2), \quad (3.13)$$

in which the pure second derivatives cancel between the two probes. Theorem 3.2 adds to (3.13) the information that two evaluations are not merely sufficient but necessary. Complex directions appear as soon as some exponent exceeds one.

REMARK 3.4 (Entire activations are essential). *The directions (3.11) are complex whenever $m_j \geq 3$ for some j , so the network is evaluated at complex arguments and its activations must admit a holomorphic extension there. The functions $\sinh$ and $\cosh$ are entire, and the recurrence (2.6) is an algebraic identity that holds verbatim on complex tensors. By contrast $\tanh$ is meromorphic, with poles at $i\pi/2 + i k\pi$ for $k \in \mathbb{Z}$, so its jet is defined only away from those points and the admissible directions depend on the activation input. Entire activations are therefore a requirement of the construction rather than an incidental convenience, and they are the reason the demonstration of Section 4 uses a $\sinh$ network for the proposed method.*

REMARK 3.5 (Scope of the rank reduction). *Theorem 3.2 concerns one monomial derivative at a time. For the square-free multi-index $a = (1, \dots, 1)$ of order p the rank (3.10) equals 2^{p-1} , so the schedule is exponentially long; since the theorem gives a minimum, this is a statement about the problem and not about the construction, and no directional formula for such a derivative can be shorter. An operator sum such as Δ^m must be expanded and evaluated term by term, and the theorem then applies to each term separately; it makes no claim of joint optimality for the sum, and a trace-collapsing method [4, 14] can be preferable when only a Laplacian is required. The stochastic Taylor derivative estimator [16] solves the strictly broader problem of an arbitrary contraction of the order- p derivative tensor, with unbiased estimates whose variance is controlled by a sampling budget; the two approaches are*

complementary, and the schedule of Theorem 3.2 is the deterministic minimum for one fixed multi-index.

Table 3.1 evaluates (3.10) for the active-exponent patterns up to $p = 6$. Because every entry is a minimum, the table reads as a statement about which derivatives are cheap to extract from directional evaluations and which are not: a pure power needs one direction at any order, a derivative that repeats every index stays modest as the order grows, and a square-free derivative is intrinsically exponential. The explicit schedules for the patterns that occur in Section 4 are listed in Appendix A.2.

TABLE 3.1

The minimum number of directional evaluations $R_C(\mathbf{z}^a)$ of (3.10), for the active-exponent patterns up to $p = 6$. Repeating an index keeps the count low; the square-free pattern (1^p) is exponential in the order.

pattern	example multi-index	R_C
(p)	$\partial_1^p u$	1
$(2, 1)$	u_{112}	3
$(1, 1, 1)$	u_{123}	4
$(3, 1)$	u_{1112}	4
$(2, 2)$	u_{1122}	3
$(2, 1, 1)$	u_{1123}	6
$(1, 1, 1, 1)$	u_{1234}	8
$(4, 2)$	u_{111122}	5
$(2, 2, 2)$	u_{112233}	9
(1^6)	u_{123456}	32

3.3. Algorithm, gradients, and cost. Algorithm 1 assembles the pieces. The schedule (3.11) is built once, its directions are stacked into a single batch, and one jet pass through the network returns the order- p coefficient for every direction at once; the reported value is then the coefficient-weighted sum. The schedule tensors depend on ν , the device, and the scalar type, but not on the trainable parameters. The reference implementation builds them at call entry and includes that work in the measured throughput of Section 4, while a caller that reuses one multi-index may cache them. For a real-valued model the conjugate pairing of the roots of unity makes the returned value real in exact arithmetic; the implementation keeps the complex tensor and takes its real part only when the target PDE is real-valued.

Algorithm 1 must be differentiable in θ , since its output enters the residual term of (2.2). Differentiating the activation recurrences (2.5)–(2.6) by reverse-mode automatic differentiation would rebuild inside the backward pass the nesting that the jet was introduced to avoid. We therefore register each activation jet as a single primitive with the closed-form adjoint

$$g_x^{(m)} = \sum_{j=0}^{p-m} \bar{q}_j g_y^{(m+j)}, \quad q(\tau) := \phi'(x(\tau)), \quad (3.14)$$

in which q_j is the j th Taylor coefficient of q and $g^{(m)}$ denotes the cotangent of the order- m coefficient. Equation (3.14) follows from the chain rule applied to the jet of ϕ and uses the convention of a real-valued loss for complex parameters, so that the conjugation is the Hermitian adjoint of multiplication by q_j . The backward pass thus costs one more triangular convolution per activation and never differentiates through the recurrence itself.

Algorithm 1 Single-monomial $\partial^\nu u(\mathbf{x})$ from a rank-optimal schedule and one batched Taylor jet.

Require: scalar network u built from supported analytic primitives; point $\mathbf{x} \in \mathbb{R}^d$; multi-index ν of order p

Ensure: the value $\partial^\nu u(\mathbf{x})$

- 1: Relabel the active coordinates of ν as $i_0, \dots, i_n$ with exponents $1 \leq a_0 \leq \dots \leq a_n$.
 - 2: Set $m_j := a_j + 1$ for $j = 1, \dots, n$ and $\mathcal{I} := U_{m_1} \times \dots \times U_{m_n}$.
 - 3: Initialize $\mathcal{S} \leftarrow \emptyset$.
 - 4: **for** $\zeta = (\zeta_1, \dots, \zeta_n) \in \mathcal{I}$ **do**
 - 5: Set $\mathbf{v}_\zeta \leftarrow e_{i_0} + \sum_{j=1}^n \zeta_j e_{i_j}$ and $c_\zeta \leftarrow \nu! (\prod_{j=1}^n m_j)^{-1} \prod_{j=1}^n \zeta_j$.
 - 6: Update $\mathcal{S} \leftarrow \mathcal{S} \cup \{(c_\zeta, \mathbf{v}_\zeta)\}$.
 - 7: **end for**
 - 8: Stack the directions of $\mathcal{S}$ into $V_\mathcal{S} \in \mathbb{C}^{|\mathcal{S}| \times d}$.
 - 9: Form the batched input jet $\mathbf{J}_0 \leftarrow (\mathbf{x} \text{ replicated}, V_\mathcal{S}, 0, \dots, 0)$.
 - 10: Propagate $\mathbf{J}_0$ through u by (2.4)–(2.6) to obtain $\mathbf{J}_L$.
 - 11: Set $t \leftarrow [\mathbf{J}_L]_p$, one order- p coefficient per direction.
 - 12: **return** $\sum_{r=1}^{|\mathcal{S}|} c_r t_r$.
-

The cost of Algorithm 1 separates into a per-direction and a per-schedule factor. For one direction, the jet stores $p + 1$ coefficient tensors per activation, an affine layer applies W to each of them, and the activation recurrences require $\sum_{k=1}^p O(k) = O(p^2)$ elementwise products per scalar. The total work and the directional storage then scale linearly in the schedule length $R_C(\mathbf{z}^a)$, which by Theorem 3.2 is as small as an exact directional formula can be. The rank reduction is an exact statement about that factor and not about wall-clock time, which also depends on complex arithmetic, kernel launch overhead, memory traffic, and whether parameter gradients are required; for small networks or small batches these terms can dominate. Section 4 therefore reports measured throughput alongside accuracy.

Two properties of the construction bear on its numerical behaviour. First, the schedule coefficients are known in closed form, so no ill-conditioned linear system is solved to obtain them, and the method has neither a finite-difference step size nor a Monte Carlo variance. Second, the remaining error is ordinary floating-point round-off from the Taylor recurrences, from complex arithmetic, and from the final summation over directions. Avoiding a linear solve does not by itself eliminate cancellation, factorial growth of the coefficients, or overflow of $\sinh$ at large arguments; the verification of Section 4.1 covers double precision through order six and is not a general conditioning study.

4. Numerical experiments. This section asks two questions about the backend and nothing else. Does it compute the derivative it claims to compute, and does the reduction in directional evaluations turn into a reduction in cost? The first is settled against nested automatic differentiation in double precision, with no PDE and no training involved. The second requires a workload, because a shorter schedule is an algebraic statement whose effect on running time also depends on complex arithmetic, batching, and the backward pass; we therefore run complete training loops in which the derivative backend is the only thing that varies, and measure the time and the memory one optimizer step costs. Physics-informed training is the workload rather

than the subject: it is a demanding and reproducible consumer of high-order derivatives, and every quantity below is a property of the backend, not of an architecture.

4.1. Verification of the derivative backend. All checks use double precision and are executable tests in the repository. The roots-of-unity coefficient filter of Lemma A.1 is evaluated against every degree-four monomial in five variables for the active-exponent pattern $(2, 1, 1)$, with a maximum admissible absolute coefficient error of 10^{-12} . The length of the schedule that the implementation builds is checked against the closed-form rank (3.10) for every pattern of Table 3.1.

The two jet backends are then compared with nested automatic differentiation on real-parameter networks using the tanh, logistic, sin, and sinh activations, for the patterns $(2, 2)$, $(1, 1, 1)$, and (3) . Values must agree to relative tolerance 2×10^{-10} and absolute tolerance 2×10^{-11} , and the imaginary remainder of a real target must not exceed 2×10^{-11} . Gradients of a squared derivative loss with respect to every network parameter are compared in the same way, with tolerances 2×10^{-9} relative and 2×10^{-11} absolute, for both real- and complex-parameter networks; additional analytic checks cover the orders 1, 2, 3, 4, and 6 for a scalar complex sinh network. Every check passes at the tolerance stated for it. These tests establish that the implementation computes what Theorem 3.2 predicts; they do not turn a reduction in direction count into a hardware-independent timing claim.

4.2. A controlled comparison of derivative backends. The remaining study isolates the derivative backend. Protocol `jsc_v3` runs two lines that share the surrogate, the loss, the optimizer, the collocation points, the evaluation rule, the seeds, and the wall-clock budget, and differ in one respect: how each monomial derivative of the residual is evaluated. The first line uses Algorithm 1, that is, the rank-optimal complex schedule evaluated by one batched Taylor jet, and is labelled *jet* below. The second evaluates the same derivative of the same network by nested reverse-mode automatic differentiation, with the graph retained after every coordinate derivative, and is labelled *nested*. Both lines are the same complex-parameter sinh network with four hidden layers of width $H = 128$, hence the same 100,098 trainable real degrees of freedom, the same first-layer scale, and the same `complex128` arithmetic. Both use Adam with initial learning rate 10^{-3} and cosine decay, 4096 fixed interior points, 512 fixed boundary points, paired per-seed collocation, the same per-setting boundary weights, and a common held-out set of 8192 points. Each line is run for three seeds with a budget of 1000 seconds per run.

A *setting* is one problem together with one value of its sweep parameter, and the protocol covers nine: the two-dimensional polyharmonic problem at orders $2m \in \{2, 4, 6\}$, the radial chirp at $a \in \{1, 2, 3\}$, and the time-harmonic Maxwell problem at $a \in \{2, 4, 6\}$, all defined as in Section 4.3. All 54 runs were produced by one source commit on one NVIDIA H20 GPU with PyTorch 2.5.1 and CUDA 12.1, so unlike a comparison assembled from separate campaigns, timings here are comparable across settings as well as within one. Cost is the measured time per optimizer step together with the peak memory allocated by a run, and accuracy is the held-out relative L^2 error

$$\mathcal{E}_{L^2} := \frac{\|u_\theta - u_\star\|_{L^2(\Omega)}}{\|u_\star\|_{L^2(\Omega)}}, \quad (4.1)$$

evaluated on that held-out set against the exact solution $u_\star$. A setting enters the tables only after automated checks confirm all six method-seed rows, finite metrics, nonempty monotone histories, and the protocol metadata.

4.3. Benchmark problems. The three families place different demands on the derivative. For the polyharmonic problem on $(-1, 1)^2$ we solve

$$\Delta^m u = (-2\pi^2)^m u_\star \quad \text{in } (-1, 1)^2, \quad u_\star(\mathbf{x}) = \sin(\pi x_1) \sin(\pi x_2), \quad (4.2)$$

at the orders $2m \in \{2, 4, 6\}$, with the Navier boundary targets $u = u_\star$ and $\Delta^j u = (-2\pi^2)^j u_\star$ for $j = 1, \dots, m-1$, all of which vanish for this $u_\star$. This family is the reason the derivative order appears as a variable: expanding Δ^m in two dimensions produces the patterns $(2m)$ and, for $m \geq 2$, the mixed patterns whose schedules Table 3.1 lists. At $2m = 4$ the mixed term is $\partial_1^2 \partial_2^2$, which (3.11) evaluates with three directions; at $2m = 6$ the mixed terms are $\partial_1^4 \partial_2^2$ and $\partial_1^2 \partial_2^4$, which take five each.

The other two families sit at order two and sweep a frequency instead. On $(-1, 1)^2$ the radial chirp is

$$-\Delta u + u = f, \quad u_\star(x, y) = \sin\left(\frac{a\pi}{2}(x^2 + y^2)\right), \quad (4.3)$$

with Dirichlet data from $u_\star$, $a \in \{1, 2, 3\}$, and the manufactured source $f = (a\pi)^2 r^2 \sin \phi - 2a\pi \cos \phi + \sin \phi$, where $\phi := a\pi(x^2 + y^2)/2$ and $r^2 := x^2 + y^2$. Its local radial wavenumber $|\nabla \phi| = a\pi r$ varies over the domain, so the target is not a separable Fourier mode. For a two-dimensional transverse-magnetic mode, Maxwell's equations reduce to a scalar complex Helmholtz equation for the out-of-plane field E ,

$$\Delta E + \kappa^2 E = f, \quad \kappa^2 = (a\pi)^2(1 + i\beta), \quad E_\star = \exp(i a\pi(x + y)), \quad (4.4)$$

with $\beta = 0.2$, exact Dirichlet data, $a \in \{2, 4, 6\}$, and $f = [-2(a\pi)^2 + \kappa^2]E_\star$. The Maxwell family is included because its solution is complex-valued, which the schedule of Theorem 3.2 accommodates without modification: the surrogate is already evaluated at complex directions, so a complex target costs nothing extra. Since both families are of order two, every monomial term of their residuals is a pure power, which the schedule evaluates with a single direction; they therefore isolate what the Taylor jet contributes on its own, with no rank reduction involved.

TABLE 4.1

Cost of one optimizer step under the jsc-v3 protocol, which changes only the derivative backend. Times are the three-seed mean of milliseconds per step and memory is the peak allocation of a run; the ratio columns give nested automatic differentiation divided by the Taylor jet, so larger is better for the jet.

setting	p	ms per step			peak memory (MiB)		
		jet	nested	ratio	jet	nested	ratio
Polyharmonic, $2m = 2$	2	62.3	91.1	1.46	509	789	1.55
Polyharmonic, $2m = 4$	4	251.6	565.9	2.25	1759	5400	3.07
Polyharmonic, $2m = 6$	6	854.2	3496.1	4.09	5728	36976	6.46
Radial chirp, $a = 1$	2	72.8	101.7	1.40	548	852	1.55
Radial chirp, $a = 2$	2	73.4	101.9	1.39	548	852	1.55
Radial chirp, $a = 3$	2	73.4	102.1	1.39	548	852	1.55
Maxwell, $a = 2$	2	73.3	102.1	1.39	573	853	1.49
Maxwell, $a = 4$	2	73.2	101.9	1.39	573	853	1.49
Maxwell, $a = 6$	2	72.9	101.8	1.40	573	853	1.49

4.4. Cost of one derivative evaluation. Table 4.1 and Figure 4.1 report the cost of one optimizer step. At order two the jet is faster by a factor between 1.39 and

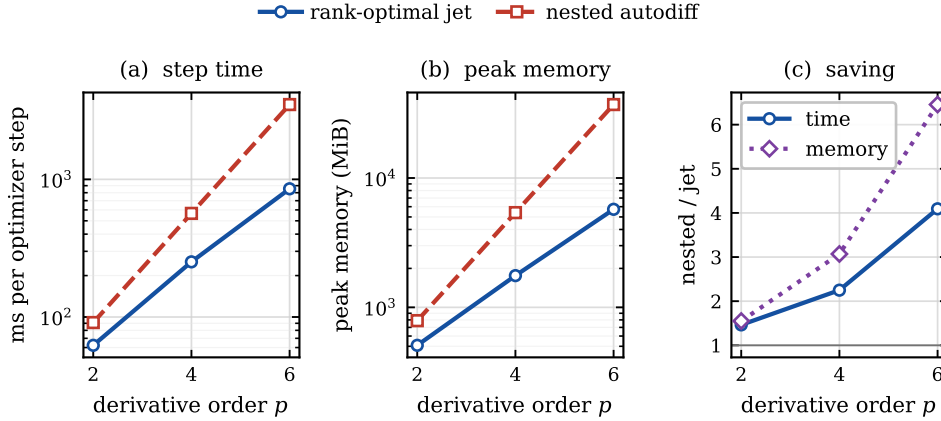


FIG. 4.1. Cost of one optimizer step against the derivative order, on the polyharmonic family, for the two backends of Section 4.2. Panels (a) and (b) show the three-seed mean step time and the peak memory of a run; panel (c) shows the ratio of nested automatic differentiation to the Taylor jet for both quantities.

1.46 across all seven order-two settings, and it allocates between 1.49 and 1.55 times less memory. This is the contribution of Taylor-mode evaluation alone, since a pure power needs one direction and no rank reduction is available; nested differentiation pays for retaining a graph across two coordinate derivatives, and the jet does not. The saving then grows with the order, exactly where the rank reduction begins to apply: at order four the step time falls by a factor of 2.25 and the peak memory by 3.07, and at order six by 4.09 and 6.46. In absolute terms, one step of the sixth-order problem costs 854 milliseconds and 5.6 gibibytes with the jet, against 3.5 seconds and 36.1 gibibytes with nested differentiation.

Two mechanisms compound in that growth, and the design of the study separates them. The jet replaces a graph of depth proportional to the derivative order and the network depth by $p + 1$ coefficient tensors per activation, which is what the memory column measures; this part is available at every order and is what the order-two settings show. The schedule then fixes how many times that jet must be run, at the minimum of Table 3.1: one direction for each pure power at any order, three for the mixed term at order four, five for each mixed term at order six. Nested differentiation has no comparable lever, since it must traverse the retained graph once per coordinate derivative of every term, which is why the ratio in panel (c) of Figure 4.1 widens with the order. Neither mechanism is a claim about asymptotics: both counts are exact, and what Table 4.1 adds is that on this hardware the algebraic reduction survives contact with complex arithmetic, kernel launches, and the backward pass.

4.5. What the saving buys. A cheaper step is only useful if the training run does more with it. Table 4.2 therefore records how many optimizer steps each line completes inside the common budget and the accuracy it reaches, and Figure 4.2 shows the trajectories. The step counts follow the cost directly: at order six the jet completes 1171 steps against 287, and at order four 3975 against 1767. The accuracy follows the step counts where the problem is hard enough for the extra steps to matter. At order four the jet reaches 6.55×10^{-4} against 1.43×10^{-3} , and at order six 3.36×10^{-2} against 9.18×10^{-2} , in both cases with the seed spread of Table 4.2 well inside the

TABLE 4.2

What the saved time buys under the same protocol: optimizer steps completed inside the common 1000-second budget, as a three-seed mean, and the held-out relative L^2 error reached, as a three-seed mean $\pm$ sample standard deviation. The lower error in each row is set in bold.

setting	steps completed		relative L^2 error	
	jet	nested	jet	nested
Polyharmonic, $2m = 2$	16058	10973	$3.25 \pm 1.15 \times 10^{-4}$	$5.42 \pm 0.88 \times 10^{-4}$
Polyharmonic, $2m = 4$	3975	1767	$6.55 \pm 0.71 \times 10^{-4}$	$1.43 \pm 0.10 \times 10^{-3}$
Polyharmonic, $2m = 6$	1171	287	$3.36 \pm 1.81 \times 10^{-2}$	$9.18 \pm 3.62 \times 10^{-2}$
Radial chirp, $a = 1$	13730	9835	$3.84 \pm 3.68 \times 10^{-4}$	$7.05 \pm 3.55 \times 10^{-4}$
Radial chirp, $a = 2$	13617	9816	$8.93 \pm 10.97 \times 10^{-4}$	$8.36 \pm 4.56 \times 10^{-4}$
Radial chirp, $a = 3$	13629	9796	$1.86 \pm 0.23 \times 10^{-4}$	$3.38 \pm 1.25 \times 10^{-4}$
Maxwell, $a = 2$	13639	9795	$1.24 \pm 0.44 \times 10^{-3}$	$1.61 \pm 0.72 \times 10^{-3}$
Maxwell, $a = 4$	13665	9814	$1.24 \pm 0.40 \times 10^{-3}$	$1.48 \pm 0.67 \times 10^{-3}$
Maxwell, $a = 6$	13712	9825	$9.27 \pm 1.52 \times 10^{-3}$	$9.42 \pm 1.49 \times 10^{-3}$

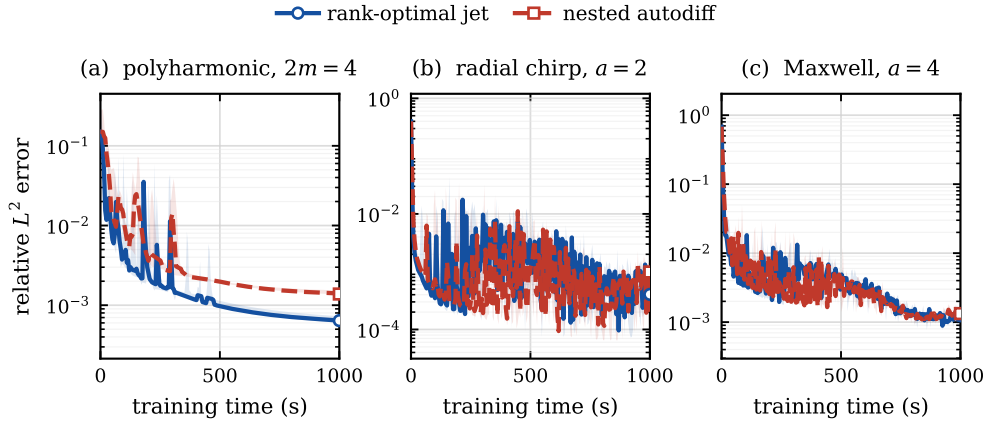


FIG. 4.2. Held-out relative L^2 error against training time for the two backends, on one representative setting of each family. Curves are the three-seed median and the shaded band is the seedwise range. Each panel is framed on the median curves.

gap. Across the nine settings the jet has the lower three-seed mean error in eight; the exception is the chirp at $a = 2$, where the two lines are within one seed standard deviation of each other.

The order-two settings are the honest counterweight. There the extra steps buy little: the two backends track each other in panels (b) and (c) of Figure 4.2, and in five of the six frequency-swept settings the two means differ by less than one seed standard deviation. A cheaper derivative is not by itself a better optimizer, and at low order the derivative is not what limits the run. What the comparison does establish is the claim the method makes: for one prescribed mixed derivative the schedule is exact and its cost advantage grows with the derivative order, in wall time and in memory, without touching the model, the loss, or the optimizer.

5. Conclusion. Evaluating one prescribed high-order mixed derivative exactly is a question about how few directions suffice, and we have shown that it is the Waring problem for the associated monomial. The identification is an equivalence rather than

a bound: a directional formula of a given length exists precisely when the monomial decomposes into that many powers of linear forms. The rank formula of Carlini, Catalisano, and Geramita then evaluates the minimum over the complex numbers as the product of the incremented active exponents, and a roots-of-unity construction attains it in closed form, with coefficients known in advance rather than obtained from a linear solve. Because the answer is a minimum, it also delimits itself: it says where a shorter formula is impossible, and the elementary two-point identity for a second mixed derivative is recovered as its smallest instance.

Combined with a batched Taylor jet through an entire activation and a closed-form vector-Jacobian product, the schedule becomes a derivative backend that is algebraically exact and differentiable in the model parameters. Double-precision tests reproduce nested automatic differentiation for values and for parameter gradients through order six. Measured against nested differentiation in training runs where nothing else varies, the backend reduces the time of one optimizer step by factors of about 1.4, 2.3, and 4.1 at derivative orders two, four, and six, and its peak memory by about 1.5, 3.1, and 6.5; on the higher-order problems the steps thus gained lower the error reached within a fixed budget. The two mechanisms behind that growth are separable in the measurements: Taylor-mode evaluation removes the nested graph at every order, and the rank reduction shortens the schedule wherever an index repeats.

The schedule is shortest for repeated-index multi-indices and exponentially long for square-free ones, which by the same theorem is a property of the problem rather than of the method. Operator sums are handled term by term, so the construction carries no joint optimality claim for a sum, and trace-collapsing or stochastic methods remain preferable for a Laplacian or for a broad tensor contraction. At low derivative order a cheaper derivative also buys little in a complete run, since the derivative is not what limits it. Natural continuations are a rank-aware expansion of an operator sum that reuses directions across its terms, a conditioning analysis of the schedule at high order and in reduced precision, and the extension of the same apolarity argument to derivative tensors contracted against a fixed structured family.

Appendix A. Schedules, filters, and the apolar bound.

This appendix collects the roots-of-unity filter used in the proof of Theorem 3.2, the explicit schedules for the derivative patterns that occur in Section 4, and the apolarity argument behind the rank formula (3.10). The formulas of Appendix A.2 are those the implementation evaluates.

A.1. Roots-of-unity filter. LEMMA A.1. *For every integer $m \geq 1$ and every integer k ,*

$$\sum_{\zeta \in U_m} \zeta^k = m \mathbf{1}_{m|k}. \quad (\text{A.1})$$

Proof. Let $\omega := \exp(2\pi i/m)$, so that every element of U_m is ω^ℓ for exactly one $\ell \in \{0, \dots, m-1\}$ and

$$\sum_{\zeta \in U_m} \zeta^k = \sum_{\ell=0}^{m-1} (\omega^k)^\ell.$$

If $m \mid k$, then $\omega^k = 1$ and the sum equals m . If $m \nmid k$, then $\omega^k \neq 1$, and the finite

geometric series gives

$$\sum_{\ell=0}^{m-1} (\omega^k)^\ell = \frac{1 - \omega^{km}}{1 - \omega^k} = 0,$$

since $\omega^m = 1$. $\square$

A.2. Explicit low-order schedules. The mixed terms of the polyharmonic residual (4.2) and the pure powers of the chirp and Maxwell residuals use the following instances of (3.11).

For a pure power $\partial^\nu = \partial_1^p$ we have $n = 0$, the schedule has one direction, and T_p is evaluated once,

$$\partial_1^p u(\mathbf{x}) = p! T_p(\mathbf{x}; e_1). \quad (\text{A.2})$$

For the fourth-order mixed term $\partial^\nu = \partial_1^2 \partial_2^2$ we have $n = 1$, $a = (2, 2)$, and $m_1 = 3$; with $\omega := \exp(2\pi i/3)$ and $U_3 = \{1, \omega, \omega^2\}$, formula (3.11) gives the three directions

$$\partial_1^2 \partial_2^2 u(\mathbf{x}) = \frac{4}{3} \sum_{\zeta \in U_3} \zeta T_p(\mathbf{x}; e_1 + \zeta e_2). \quad (\text{A.3})$$

For the sixth-order mixed term $\partial^\nu = \partial_1^2 \partial_2^2 \partial_3^2$ we have $n = 2$, $a = (2, 2, 2)$, and $m_1 = m_2 = 3$, so (3.11) gives $R_C(\mathbf{z}^a) = 9$ directions,

$$\partial_1^2 \partial_2^2 \partial_3^2 u(\mathbf{x}) = \frac{8}{9} \sum_{\zeta_1, \zeta_2 \in U_3} \zeta_1 \zeta_2 T_p(\mathbf{x}; e_1 + \zeta_1 e_2 + \zeta_2 e_3). \quad (\text{A.4})$$

The equivalent polynomial identity (3.9) for (A.4) reads

$$6! z_0^2 z_1^2 z_2^2 = \frac{8}{9} \sum_{\zeta_1, \zeta_2 \in U_3} \zeta_1 \zeta_2 (z_0 + \zeta_1 z_1 + \zeta_2 z_2)^6, \quad (\text{A.5})$$

and it can be checked directly: expanding the right-hand side and collecting the coefficient of $z_0^{\beta_0} z_1^{\beta_1} z_2^{\beta_2}$ produces the product $\prod_{j=1}^2 \sum_{\zeta_j \in U_3} \zeta_j^{1+\beta_j}$, which by Lemma A.1 is nonzero only when $\beta_j \equiv 2 \pmod{3}$ for $j = 1, 2$. Together with $\beta_0 + \beta_1 + \beta_2 = 6$ this forces $\beta = (2, 2, 2)$, and the surviving constant is $\frac{8}{9} \cdot \frac{6!}{2!2!2!} \cdot 9 = 720 = 6!$.

A.3. Apolar form of the rank bound. The lower bound in (3.10) comes from apolarity, which we recall for completeness. For a homogeneous $F \in \mathbb{C}[\mathbf{z}]_p$, let $F^\perp$ denote its apolar ideal, that is, the ideal of differential operators annihilating F , and call a finite subscheme $X \subset \mathbb{P}^n$ apolar to F when $I_X \subset F^\perp$. The Waring rank then admits the characterization

$$R_C(F) = \min\{\deg X : X \subset \mathbb{P}^n \text{ finite, reduced, and apolar to } F\}. \quad (\text{A.6})$$

For the monomial $F = \mathbf{z}^a$ the apolar ideal is the complete intersection

$$(\mathbf{z}^a)^\perp = (\partial_0^{a_0+1}, \dots, \partial_n^{a_n+1}), \quad (\text{A.7})$$

and the Hilbert function of the quotient $\mathbb{C}[\partial]/(\mathbf{z}^a)^\perp$ bounds the degree of an apolar subscheme from below by the product $\prod_{j=1}^n m_j$ of (3.10); see [11, 13] for the general theory and [2] for the monomial case. The construction (3.11) exhibits a reduced apolar subscheme of exactly that degree, so the bound is attained.

REFERENCES

- [1] JESSE BETTENCOURT, MATTHEW J. JOHNSON, AND DAVID DUVENAUD, *Taylor-mode automatic differentiation for higher-order derivatives in JAX*, in NeurIPS Workshop on Program Transformations for Machine Learning, 2019.
- [2] ENRICO CARLINI, MARIA VIRGINIA CATALISANO, AND ANTHONY V. GERAMITA, *The solution to the Waring problem for monomials and the sum of coprime monomials*, Journal of Algebra, 370 (2012), pp. 5–14.
- [3] PAO-HSIUNG CHIU, JIAN CHENG WONG, CHINCHUN OOI, MY HA DAO, AND YEW-SOON ONG, *CAN-PINN: A fast physics-informed neural network based on coupled-automatic-numerical differentiation method*, Computer Methods in Applied Mechanics and Engineering, 395 (2022), p. 114909.
- [4] NICHOLAS GAO, JONAS KÖHLER, AND ADAM FOSTER, *folx: Forward Laplacian for JAX*, 2023. Software.
- [5] ANDREAS GRIEWANK AND ANDREA WALTHER, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2nd ed., 2008.
- [6] JIEQUN HAN, ARNULF JENTZEN, AND WEINAN E, *Solving high-dimensional partial differential equations using deep learning*, Proceedings of the National Academy of Sciences, 115 (2018), pp. 8505–8510.
- [7] DI HE, SHANDA LI, WENLEI SHI, XIAOTIAN GAO, JIA ZHANG, JIANG BIAN, LIWEI WANG, AND TIE-YAN LIU, *Learning physics-informed neural networks without stacked back-propagation*, in Proceedings of the 26th International Conference on Artificial Intelligence and Statistics, vol. 206 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 3034–3047.
- [8] ZHEYUAN HU, ZEKUN SHI, GEORGE EM KARNIADAKIS, AND KENJI KAWAGUCHI, *Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 424 (2024), p. 116883.
- [9] ———, *Bias-variance trade-off in physics-informed neural networks with randomized smoothing for high-dimensional PDEs*, Journal of Computational Physics, 521 (2025), p. 113524.
- [10] ZHEYUAN HU, KHEMRAJ SHUKLA, GEORGE EM KARNIADAKIS, AND KENJI KAWAGUCHI, *Tackling the curse of dimensionality with physics-informed neural networks*, Neural Networks, 176 (2024), p. 106369.
- [11] ANTHONY IARROBINO AND VASSIL KANEV, *Power Sums, Gorenstein Algebras, and Determinantal Loci*, vol. 1721 of Lecture Notes in Mathematics, Springer, Berlin, 1999.
- [12] GEORGE EM KARNIADAKIS, IOANNIS G. KEVREKIDIS, LU LU, PARIS PERDIKARIS, SIFAN WANG, AND LIU YANG, *Physics-informed machine learning*, Nature Reviews Physics, 3 (2021), pp. 422–440.
- [13] JOSEPH M. LANDSBERG, *Tensors: Geometry and Applications*, vol. 128 of Graduate Studies in Mathematics, American Mathematical Society, Providence, RI, 2012.
- [14] RUICHEN LI, HAOTIAN YE, DU JIANG, XUELAN WEN, CHUWEI WANG, ZHE LI, XIANG LI, DI HE, JI CHEN, WEILUO REN, AND LIWEI WANG, *A computational framework for neural network-based variational Monte Carlo with Forward Laplacian*, Nature Machine Intelligence, 6 (2024), pp. 209–219.
- [15] MAZIAR RAISSI, PARIS PERDIKARIS, AND GEORGE E. KARNIADAKIS, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378 (2019), pp. 686–707.
- [16] ZEKUN SHI, ZHEYUAN HU, MIN LIN, AND KENJI KAWAGUCHI, *Stochastic Taylor derivative estimator: Efficient amortization for arbitrary differential operators*, Advances in Neural Information Processing Systems (NeurIPS), 37 (2024), pp. 122316–122353.
- [17] CHARLES M. STEIN, *Estimation of the mean of a multivariate normal distribution*, The Annals of Statistics, 9 (1981), pp. 1135–1151.